

CAN-Bus Attack Detection With Deep Learning

Flora Amato^{ID}, Luigi Coppolino^{ID}, Francesco Mercaldo^{ID}, Francesco Moscato^{ID},
Roberto Nardone^{ID}, and Antonella Santone^{ID}

Abstract—Modern cars include a huge number of sensors and actuators, which continuously exchange data and control commands. The most used protocol for communication of different components in automotive system is the Controller Area Network (CAN). According to CAN, components communicate by broadcasting messages on a bus. In addition, the standard definition of the protocol does not provide information for authentication, so exposing it to attacks. This paper proposes a method based on deep learning aiming at discovering attacks towards the CAN-bus. In particular, Neural Networks and MultiLayer Perceptrons are the class of networks employed in our approach. We also validate our approach by analysing a real-world dataset with the injection of messages from different types of attacks: denial of service, fuzzy pattern attacks, and attacks against specific components. The obtained results are encouraging and demonstrate the effectiveness of the approach.

Index Terms—Automotive, intelligent systems, deep learning, neural networks, attack detection, security.

I. INTRODUCTION

IN THE last years, the automotive domain has experienced a rapid growth of complexity, mainly due to the introduction of more and more complex components for control, supervision or driving assistance. Modern cars include nowadays many electronic components, computing units and software as well. In addition, all components are linked each other by complex networks. These elements work no more “stand-alone”, but they need to exchange a plethora of information, generating a large amount of data to manage [1].

Modern cars include dozens of independent processing units — the so-called Electronic Control Units (ECUs) — that are interconnected by different buses or networks. [2], [3].

Manuscript received March 30, 2020; revised September 22, 2020; accepted November 23, 2020. Date of publication January 15, 2021; date of current version August 9, 2021. The Associate Editor for this article was M. Guizani. (Corresponding author: Flora Amato.)

Flora Amato is with the Department of Electrical and Information Technology Engineering (DIETI), University of Naples Federico II, 80125 Napoli, Italy (e-mail: flora.amato@unina.it).

Luigi Coppolino is with the Department of Engineering, University of Naples Parthenope, 80133 Napoli, Italy (e-mail: luigi.coppolino@uniparthenope.it).

Francesco Mercaldo is with the Institute for Informatics and Telematics, National Research Council of Italy (CNR), 56124 Pisa, Italy, and also with the Department of Medicine and Health Sciences “Vincenzo Tiberio,” University of Molise, 86100 Campobasso, Italy (e-mail: francesco.mercaldo@iit.cnr.it).

Francesco Moscato is with the Department of Computer Engineering, Electrical Engineering and Applied Mathematics (DIEM), University of Salerno, 84084 Fisciano (SA), Italy (e-mail: fmoscato@unisa.it).

Roberto Nardone is with the Department of Information Engineering, Infrastructure and Sustainable Energy (DIIES), Mediterranean University of Reggio Calabria, 89124 Reggio Calabria, Italy (e-mail: roberto.nardone@unirc.it).

Antonella Santone is with the Department of Medicine and Health Sciences “Vincenzo Tiberio,” University of Molise, 86100 Campobasso, Italy (e-mail: antonella.santone@unimol.it).

Digital Object Identifier 10.1109/TITS.2020.3046974

ECUs execute many safety controls, such as skid detection, crash prediction or anti-lock braking [1]. In general, the whole control system, so considerably changed during the years, has soft or hard on real-time requirements on network components in order to guarantee the correct operation and the sufficient quality levels (i.e., performance, reliability, safety) of the whole system.

The most used communication standard is the Controller Area Network (CAN) (also known as CAN-bus) [3]. The CAN standard defines a communication protocol; it dates back to the 80s and it was developed by Bosch GmbH company in order to provide efficient communication among several automotive applications, also with a critical impact on the safety. It became a standard within the ISO 11898-1:2011.¹ The CAN standard was initially applied to automotive, but it is currently employed in many industrial applications for communication of embedded components. It mainly consists in a broadcast protocol where messages sent by a sender are analysed by all receivers connected to the same network, which in turn decide if the message have to be delivered at application layer [4].

In the definition of the CAN standard, there is no built-in security mechanism. This exposes modern cars to cyber-security threats. As an example, it is very simple to sniff packets from a CAN-bus, as well as it is easy to enact impersonation attack. In particular, as it will be shown in the following, malicious ECU can steal identities injecting packets with identifiers of other components. However, the application of complex reverse engineering techniques is needed to analyse traffic since every manufacturer and every car model develop a different implementation of the CAN standard (which is not publicly disclose), which differs in the customization of IDs to represent data, in the data format, etc [2].

With the wide adoption of CAN in modern cars, passengers are continuously exposed to threats due to attacks to code running on their automobiles. Indeed, in these systems a compromised software could lead to relevant injuries for passengers [5], [6].

This paper represents a step forward in the direction of securing CAN-buses by means of Artificial Intelligence techniques. In particular, it shows that deep learning is an effective technique that can be exploited to detect attacks on CAN-bus. In fact, the paper proposes a method for identification of attacks based on deep learning, where data from packets represent the features vector. The idea relies on the fact that the attacks targeting CAN are characterized by a common attempt to compromise well-chosen mechanisms in

¹<https://www.iso.org/standard/63648.html>

the protocol [7]. A concrete experimentation is also conducted by using a real-world dataset where the goal is to prove how the proposed solution is able to discover malicious messages injected by attackers.

The rest of the paper is structured as follows. After the description of the concepts representing the background given in Section II, Section III conducts the analysis of the related works. Section IV describes the details about the proposed approach, detailing the detection technique and describing the chosen features. Then Section V describes the experimentation campaign conducted on a real-world dataset. Section VI ends the paper by drawing some final remarks and addressing possible future works.

II. BACKGROUND

A. Controller Area Network

CAN is a network standard that uses a serial protocol for messages exchanges. It was officially introduced in 2011 by standard ISO 11898-1:2011.² This standard gives all details needed to set up a connection among ECUs. CAN supports distributed real-time communication and it is widely used in communications among control and monitoring subsystems in cars. The same standard also defines two kinds of formats for frames: the Classical frame and the Flexible Data Rate Frame. In particular, the former has a payload of 8bits with a bit rate of 1 Mbps when the distance does not exceed 40 meters, it is reduced to 125 kbps when the distance increases up to 500 metres. The payload and the bitrate can be increased with the use of Flexible Data Rate frame format. At last, the standard also defines hierarchically the layers of the elements in the CAN architecture. It supports three possible implementations: (1) classical CAN frame format only; (2) classical CAN with tolerance of flexible format; (3) mixed frame formats. The last option is highly recommended for novel implementations. As a matter of fact, we can consider the protocol at layer 2 of ISO/OSI stack.

The use of CAN-bus well solves the problem of management of the increasing number of sensors and control devices in modern cars, that previously needed point-to-point connections. CAN ensures low design and implementation costs in concrete applications and in hostile environments (i.e., high noise and disturbs in the communications) [1].

In details, the standard defines that CAN packets have either 11 bits or 29 bits-long identifiers. To the best of our knowledge, automotive applications typically use 11 bits identifiers. Data have a length up to 8 bytes. Optionally, a checksum could be added but only if it fits the maximum length of 8 bytes (indeed the standard does not specify this). As an example, Toyota cars exploit checksums with a variable length of the data frame, while many other manufacturers prefer data frame o. fixed length with or without checksums [8]. Usually, CAN packets can be classified as normal or diagnostic. In the first category, we insert all the packets sent by the ECUs, either containing information and sent in broadcast to all other ECUs or containing commands for specific ECUs. Diagnostic packets can be used to monitor the state of the network

and of connected elements. The former type of packets is continuously present in a car, even if the observation time is reduced to few milliseconds; the latter is usually not used during normal operation.

The CAN packets mainly contain the following fields:

- Identifier: unique identifier of the specific CAN message also representing message priority, expressed over 11 bits;
- Remote Transmission Request (RTR): bit specifying the type of message, choosing between data frame and remote request;
- Data Length Code (DLC): number of data bytes, varying from 0 to 8 and expressed as a 4-bit integer;
- Data Field: data to be transmitted, in a length varying from 0 to 8 bytes, according to what expressed in the DLC field;
- Cyclic Redundancy Check (CRC): checksum of the message.

The following Listing reports an example of frame format. As said before, the CAN packet contains up to 8 bytes of data.

```
IDH: 03, IDL: B1, Len: 08,
Data: 19 21 22 30 08 8E 6D 18
```

B. Deep Learning Techniques

We remember that Deep learning techniques have the characteristics of exploiting non linear processing among logical units that are connected each other in cascade over different layers, where outputs of one layer serve as input of next one. These techniques support both supervised and unsupervised approaches.

In particular, we use Deep Neural Networks (DNN), which is a neural network with multiple layers hidden between input and output layers. DNNs are able to implement non-linear classifiers and they create models by layered composition of primitives [9].

DNNs usually exploit feedforward connection of inputs and outputs [10].

In following work we consider the supervised deep learning techniques using two different classification algorithms:

- *Neural Network*: this simulates human brain where “neurons” are functions managing data in a complex network, which resembles statistical methods [9]. Neurons (perceptrons) are connected each other resembling linear regressions [11].
- *Multi Layer Perceptron (MLP)*: this is a class of feed-forward artificial neural network. They allow for implementation of space separators. They present one or more hidden layers [12]. Perceptrons evaluates outputs by combining different inputs by a linear weighted transfer function [11]. Output layer reports classification results over mapped inputs.

III. RELATED WORK

The CAN standard has been widely studied in literature. With a specific focus on the attack detection, many works

²<https://www.iso.org/standard/63648.html>

are currently present in literature. In [5] on the basis of the current literature, the authors describe the set of possible attacks against a smart car, the found weaknesses and detail the attack vector, assessing the threat, the vulnerability and the impact. Many of the reported attacks can be conducted by violating the CAN-bus. The wide range of possible attacks against the CAN-bus is also described in [13], where the authors confirm that multiple attack vectors exploit CAN vulnerabilities. In addition, also wireless communications can be exploited to take the control of the vehicle, such as positioning systems, on-board WiFi and digital radio. Wireless attacks are also at the basis of the work in [14], where the author verify how a smartphone app can be used for long-range wireless attacks.

From the protection point of view, the work in [15] proposes an approach to develop a detector of CAN attacks based on long short-term memory, which is a Recurrent Neural Network used in deep learning. This detector is mainly oriented to the prediction of the next data packet originating by an ECU by analysing the previous messages generated by the same source. The experimental validation seems to confirm the anomalies detection has a low false positive rate. The work in [16] proposes an algorithm aiming at detecting denial of service attacks continuously monitoring normal behavior of the system. Similarly, the work in [17] offers an intrusion detection system for vehicular ad-hoc networks using Artificial Neural Networks, mainly employed to detect Denial of Service (DoS) attacks. The idea behind this system is to detect an attack using the data generated during communication. The simulation has been obtained by exploiting the NS2 simulator, in order to basically generate the normal and malicious behavior. In the same direction, the paper in [18] proposes an approach based on autonomous road-side infrastructure network that includes a certification authority, distributed among the different regions. The role of the road-side infrastructures is to cache and to forward certificates, transparently from other autonomous network. The final objective is to demonstrate that intrusion detection system using certification authorities offer an higher level of protection against malicious cars with valid certificates, with better efficiency, scalability and security.

A deep convolutional neural network is proposed in [19] as can bus intrusion detection. They adopt the Inception-ResNet model which was initially developed for image classification.

CANTransfer [20] adopts transfer learning implementing a convolutional long short-term memory. CANTransfer is aimed to detect whether an attack is happening without detect the specific attacks.

The idea of adding devices in the road-side infrastructure is also exploited in the work in [21] that describes a sensor-based approach. The sensors are intelligent devices, able to detect incorrect information and also the nodes originating this information. The authors in [22] increase the amount of data to analyse, by collecting external data (radars, traffic etc.) dynamically evaluating data similarity. Authors in [23] exploit features obtained from the can bus for driver profiling. In detail they show how supervised machine learning algorithms can be employed to distinguish between the car owner and impostors.

CANet [24] represents a neural network architecture for detecting intrusions targeting the can bus. CANet is evaluated on simulated attacks i.e., (malicious and legitimate) can packets obtained by exploiting a simulated environment and not a real one. Machine learning is investigated in [25] for bus travel time prediction with the aim to provide an efficient scheduling for the passengers. Basically authors experiment three regression methods i.e., linear regression, support vector regression and artificial neural network for travel time prediction.

CANeleon [26] is aimed to detect spoofed frame ID attacks in can networks. The idea behind CANeleon is to defend the can bus from an attacker injecting malicious packets with legitimate IDs.

An intrusion detector aimed to detect spoofing attacks in connected electric vehicles is proposed in [27], where authors exploit supervised machine learning by building models with the Random Forest and the k-Nearest Neighbor algorithms.

With respect to the existing approaches, the one described in this paper adopts deep learning techniques to identify human actions within the system, in order to discriminate between normal and injected messages. The model developed in this work follows the guidelines drawn in several works where the authors use deep learning to classify human actions without using any prior knowledge [28]–[30]. In this direction, this work could be seen as a first step in the application of techniques aiming at detecting human behaviour in a complex control environment, such as the automotive domain. At last, we have to say that, to the best of our knowledge, this work is the first that performs a validation with real-world data.

IV. THE METHOD

This section describes the proposed method we developed to detect attacks targeting CAN-bus. To distinguish the messages injected by a malicious attacker from those belonging to the normal communication flow, the proposed method uses data bytes of CAN packet. With reference to the frame format described in Section II-A, features from number 1 to number 8, refer to the first to the eighth byte, and respectively to values in the Listing: 19, 21, 22, 30, 08, 8E, 6D and 18.

We validate our approach by providing a real-world use case. We use four different datasets that have been provided freely.³ Datasets include four kinds of attacks: dos attack (*dos*), fuzzy attack (*fuzzy*), drive gear spoofing (*gear*) and RPM gauge spoofing (*rpm*) [1]. Datasets have been collected by OBD-II (On-Board Diagnostics) logging during attacks. More that 300 malicious messages have been logged during different attacks that lasted from 3 to 5 second. Datasets include from 30 to 40 minutes of bus traffic.

The leftmost part of Fig.1 reports an example of Denial of Service (*dos*) attack, where the message with ‘0000’ CAN ID is injected every 0.3 milliseconds.

The rightmost part of Fig.1 shows a Fuzzy attack with CAN ID and DATA changing every 0.5 ms.

³<https://sites.google.com/a/hksecurity.net/ocslab/Datasets/car-hacking-dataset>

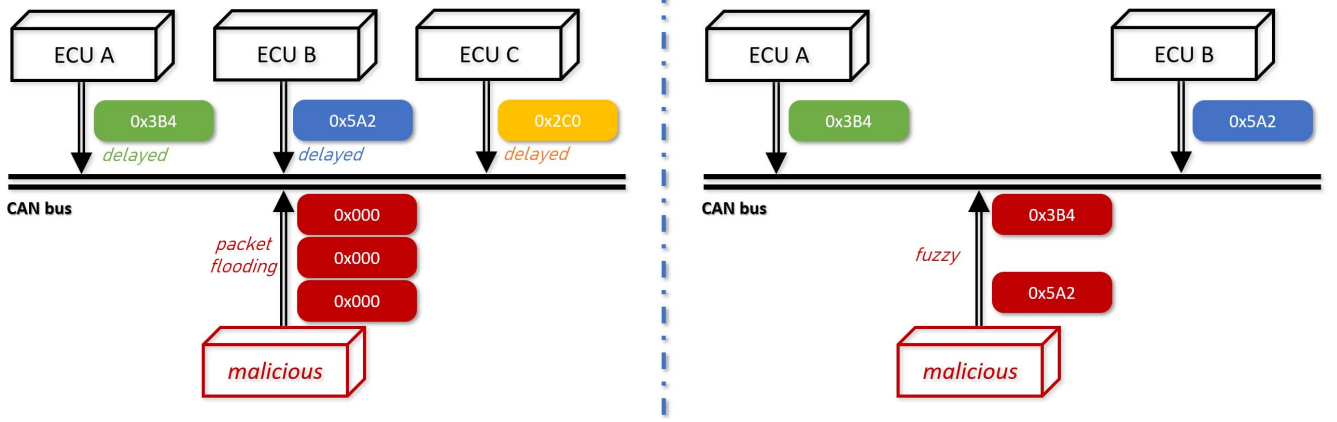


Fig. 1. The dos (in the left box) and fuzzy (in the right box) attacks.

TABLE I

NUMBER OF TOTAL, NORMAL AND INJECTED MESSAGES IN THE FOUR DATASETS WE CONSIDER FOR THE ATTACKS AND IN THE FULL DATASET, CONSTRUCTED BY ADDING ALL THE MESSAGES

attack type	messages	legitimate messages	attack messages
<i>dos</i>	3,685,771	3,088,250	597,521
<i>fuzzy</i>	3,838,860	2,759,492	1,079,368
<i>gear</i>	4,443,142	2,766,522	1,676,620
<i>rpm</i>	4,625,702	2,292,185	2,333,517
<i>full</i>	16,572,475	10,896,449	5,676,026

The *gear/rpm* attacks require the injection of messages related to a CAN ID related to gear/rpm data every 1 ms. Notice that *rpm* is really important in the automotive context, since increasing maliciously the RPMs may cause damage to engine.

Table I reports statistics about the number of messages in the different datasets.

To validate the proposed approach, we set up an experiment in order to understand if the designed features vector contains enough information to classify different attacks.

We follow three steps in validation: (i) Descriptive Analysis: we create statistical dependencies among normal and malicious messages. (ii) Hypotheses testing: we check if different distributions exist for different kinds of messages. (iii) Classification Analysis: we evaluate classifications results.

For the first point, we represent boxplots of the distribution of injected and normal messages with reference to feature vectors.

In the second step, we test the following hypothesis: H_0 : ‘normal and injected messages show closer values for feature vector’.

The null hypothesis was evaluated with Mann-Whitney, with Kolmogorov-Smirnov Test and with Wald-Wolfowitz Test. For the three test we set a p-level equal to 0.05).

The objective is to estimate the probability of wrong classification. Here, we reach a significance level of 0.05 (i.e. we consider acceptable a mistake rate of 5%).

The last step aims at checking features vector effectiveness classify malicious and normal messages. Two different deep learning classification algorithms are considered: NN and MLP with the full set of eight features we considered.

We exploited the NN and MLP implementation provided by the Weka⁴ toolsuite, an artificial intelligence library widespread in scientific research.⁵

V. THE EXPERIMENT

We evaluate the propose deep learning based technique coherently with the three steps we previously described: (i) descriptive statistics, (ii) hypotheses testing and (iii) classification analysis.

A. Descriptive Statistics

Fig. 2 shows the distributions of the attacks, and normal distributions of messages for the F1 (i.e., Packet #1), F2 (i.e., Packet #2), F3 (i.e., Packet #3) and F5 (i.e., Packet #5) features. For sake of space, we do not discuss the boxplots related to the remaining features, where similar trends are also depicted for the features F4, F6, F7 and F8, but similar consideration can be done for the remaining features.

From the analysis of the Fig. 2, we can notice that the distributions of the *fuzzy* attacks and the normal messages are similar for the Packets number 1, 2, 3 and 5, for this reason the features can be successfully employed to train a classifier with a good precision ability to discriminate between *fuzzy* attacks and *normal* messages. With regard to the Packet #1 boxplot, we highlight that the media of the *dos*, *gear* and *rpm* messages are really closer, while the media of the *fuzzy* messages is higher than the *rpm* one. This trend is shown also in Packet #2, 3 and 5. The *gear* distribution boxplot exhibits a different trend in Packet #1, 2, 3 and 5: as a matter of fact in Packet #2, Packet #3 and Packet #5 the distributions show an higher value if compared with the values shown from Packet #1. The *dos* and the *rpm* distributions are practically the same in Packet #1, Packet #2, Packet #3 and Packet #5 boxplots; same trends are shown from the *fuzzy* attack distributions. With regards to the normal distributions, we note that the media is different between different boxplots: the media in the Packet #1, Packet #3 and Packet #5 boxplots is smaller than the one

⁴<http://www.cs.waikato.ac.nz/ml/weka/>

⁵<https://github.com/Waikato/wekaDeeplearning4j>

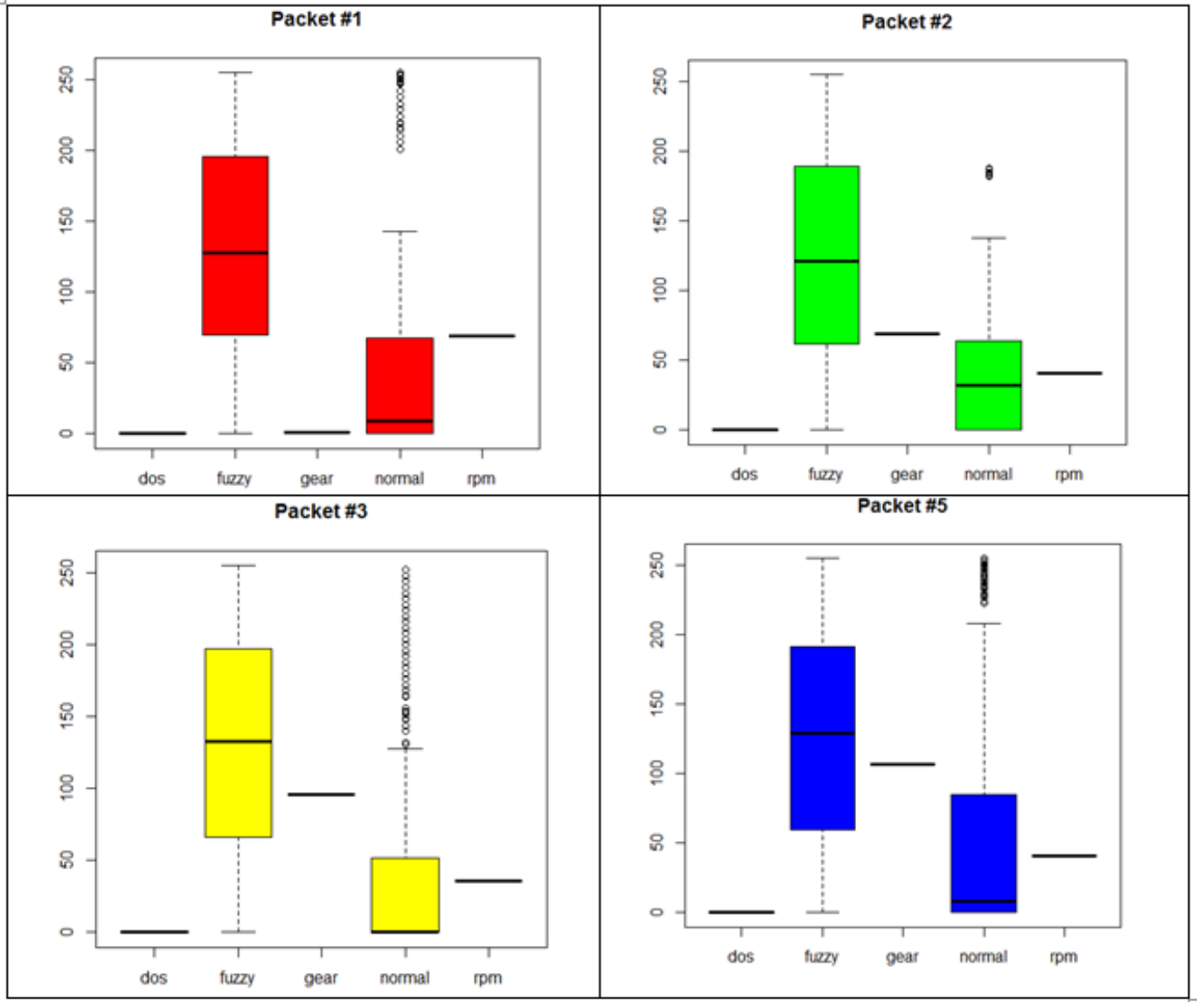


Fig. 2. The boxplots related to normal messages and the four considered attacks (i.e., *dos*, *fuzzy*, *gear* and *rpm*) distributions for the features F1 (i.e., Packet #1), F2 (i.e., Packet #2), F3 (i.e., Packet #3) and F5 (i.e., Packet #5).

reached in the Packet #2 boxplot. Clearly more the trend of different categories (i.e., *dos*, *fuzzy*, *gear*, *normal* and *rpm*) is different, more is the probability that the classifiers are able to generate models aimed to discriminate between these categories with good prediction performance).

B. Hypothesis Testing

The goal here is to understand if different distributions of normal and injected messages are identified as different with statistical evidence.

We consider valid results obtained when the null hypothesis is rejected by the three tests we are going to describe here.

Table II shows results: we can reject the null hypothesis H_0 for all features. This implies that chosen features and features vector are good to use in deep learning methods for classification.

Results show that features we selected have different values in different distributions related to the four types of messages,

confirming that features are enough and good to discriminate those classes of behaviors. As a matter of fact, considering that all features passed the *Mann-Whitney*, *Kolmogorov-Smirnov* and *Wald-Wolfowitz* Hypothesis testing, we can state that the considered feature vector can represent a good candidate to discriminate between different malicious attacks targeting the can bus. The next step i.e., the classification analysis, is aimed to build and evaluate models to evaluate the effectiveness of the features to discriminate between injected or normal packets.

C. Classification Analysis

Below we present the performance obtained by several classifiers trained with the considered feature vector. Two different task are considered: the first aimed, aimed to build a predictive model to discriminate between generic attacks and normal messages and, the second one, aimed to detect the kind of malicious attacks. For this reason for the first task a binary

TABLE II
RESULTS OF THE NULL HYPOTHESIS H_0 TEST

Features	Mann-Whitney	Kolmogorov-Smirnov	Wald-Wolfowitz	Result
F1		$p < .001$	0	<i>passed</i>
F2	0	$p < .001$	0	<i>passed</i>
F3	2.4E-04	$p < .001$	0	<i>passed</i>
F4	0	$p < .001$	0	<i>passed</i>
F5	0	$p < .001$	0	<i>passed</i>
F6	0	$p < .001$	0	<i>passed</i>
F7	0	$p < .001$	0	<i>passed</i>
F8	0	$p < .001$	0	<i>passed</i>

TABLE III
SINGLE-ATTACK CLASSIFICATION RESULTS WITH THE ONE HIDDEN LAYER MLP ALGORITHM. IM IS FOR IMPOSTOR MESSAGES, WHILE WITH NM WE LABEL THE NORMAL ONES

Category	Precision	Recall	F-Measure	MCC	Roc Area	Class
<i>dos</i>	1.000	0.972	0.986	0.963	0.985	NM
	0.955	1.000	0.977	0.963	0.985	IM
	0.983	0.982	0.982	0.963	0.985	Weighted Avg.
<i>fuzzy</i>	0.996	1.000	0.998	0.981	0.993	NM
	0.997	0.969	0.983	0.981	0.993	IM
	0.996	0.996	0.996	0.981	0.993	Weighted Avg.
<i>gear</i>	1.000	1.000	1.000	1.000	1.000	NM
	1.000	1.000	1.000	1.000	1.000	IM
	1.000	1.000	1.000	1.000	1.000	Weighted Avg.
<i>rpm</i>	1.000	1.000	1.000	1.000	1.000	NM
	1.000	1.000	1.000	1.000	1.000	IM
	1.000	1.000	1.000	1.000	1.000	Weighted Avg.

classification is considered while, with respect to the second one, we resort to a multi-label classification.

In order to analyse classification results, we consider the metrics of Precision, Recall, F-measure, MCC and RocArea metrics.

We have implemented the classifier with the features vector we described before, aiming at evaluating its accuracy.

During Training, we take into account messages (M, l), with $l \in \{IM, NM\}$. An element $m \in M$ creates a feature vector $F \in R_y$ where y is the number of the features for training phase ($y = 8$).

Then, we apply a k -fold cross-validation method: we divided the original set into k smaller sets and we used each of them as test set for the remaining $k - 1$ subsets gathered to create a training set. We iterated this process 10 times with $k = 10$. Finally, we evaluated the average accuracy from all results.

Then, we evaluated model Effectiveness by creating a new training set T and a testing set T' from the dataset D , with $T \subset D$ and $T' = D - T$. Then we first trained the classifier with T and then we tested it with T'

We used the 90% of elements in D as training dataset and the remaining elements as testing dataset.

We then iterated this process for all datasets of the four kinds of attacks we have considered, and then we finally mixed all elements in a whole dataset.

In Table III we show the results we collected by using this evaluation approach. We consider a binary classification for identification of single attacks with MLP algorithm, while Table IV shows results when using the NN algorithm. In particular we indicate with *IM* the impostor (i.e., *attack*)

messages, while with the *NM* label the legitimate (i.e., *normal*) messages.

With regard to the attack identification task, a multi label classification is considered. In this case the labels are the following: *dos* for the dos attack, *fuzzy* for the fuzzy attack, *gear* and *rpm* respectively for the gear and the rpm spoofing. Finally, Table V shows results for the multiple attacks detection.

The multilayer perceptron network we used during the identification of single attacks, is a network with one hidden layer (which Fig. 3 reports within the box labelled with “b”)

Results in Table III, report a weighted average precision that ranges from 1 (*gear* and *rpm*) to 0.983 (*dos*). The recall ranges from 1 (*gear* and *rpm*) to 0.982 (*dos*).

Evaluation of F-Measure parameter for different attack is comprised from 1 to 0.982.

NN algorithm for single attack classification uses one hidden layer too within a network of 100 units.

Table IV contains results obtained by NN detection. Notice that we have the best weighted precision while detecting *rpm* attacks. *Fuzzy* and *gear* attacks detections score a precision of 0.994 and 0.991 respectively, while *dos* attacks have a lesser precision (that anyway overcomes the value of 0.9).

For what recall evaluation concerns for NN algorithm, we can see that the best recall is for *rpm* attacks and the worst (0.886) is for *dos*. Here, the best F-Measure is 1 for *rpm*, 0.888 for *dos*, 0.991 for *gear* and 0.888 too for *fuzzy*.

These results show that both MLP and NN are (very) good detectors during single attacks detection, and that MLP scores better results compared with NN.

TABLE IV
SINGLE-ATTACK CLASSIFICATION RESULTS WITH THE NN ALGORITHM. WITH IM WE INDICATE THE
IMPOSTOR MESSAGES, WHILE WITH NM THE NORMAL ONES

Category	Precision	Recall	F-Measure	MCC	Roc Area	Class
<i>dos</i>	1.000	0.818	0.900	0.792	0.978	NM
	0.766	1.000	0.867	0.792	0.978	IM
	0.913	0.886	0.888	0.792	0.978	Weighted Avg.
<i>fuzzy</i>	0.993	1.000	0.996	0.967	0.999	NM
	0.999	0.943	0.970	0.967	0.999	IM
	0.994	0.994	0.993	0.967	0.999	Weighted Avg.
<i>gear</i>	1.000	0.988	0.994	0.973	1.000	NM
	0.958	1.000	0.979	0.973	1.000	IM
	0.991	0.991	0.991	0.973	1.000	Weighted Avg.
<i>rpm</i>	1.000	1.000	1.000	1.000	1.000	NM
	1.000	1.000	1.000	1.000	1.000	IM
	1.000	1.000	1.000	1.000	1.000	Weighted Avg.

TABLE V
SECOND EXPERIMENT RESULTS

Algorithm	Precision	Recall	F-Measure	MCC	Roc Area	Class
<i>NN</i>	0.888	0.998	0.940	0.715	0.983	normal
	0.000	0.000	0.000	0.000	0.972	dos
	0.999	0.882	0.937	0.937	0.999	fuzzy
	0.966	1.000	0.983	0.982	1.000	gear
	1.000	1.000	1.000	1.000	1.000	rpm
	0.819	0.902	0.858	0.686	0.984	Weighted Avg.
<i>MLP 0</i>	0.922	0.977	0.949	0.766	0.982	normal
	0.738	0.500	0.596	0.576	0.982	dos
	0.953	0.400	0.563	0.611	0.975	fuzzy
	0.994	1.000	0.997	0.997	1.000	gear
	0.998	1.000	0.999	0.999	1.000	rpm
	0.914	0.919	0.911	0.770	0.984	Weighted Avg.
<i>MLP 1</i>	0.998	0.957	0.977	0.912	0.990	normal
	0.739	1.000	0.850	0.844	0.982	dos
	0.991	0.956	0.973	0.972	0.991	fuzzy
	1.000	1.000	1.000	1.000	1.000	gear
	1.000	1.000	1.000	1.000	1.000	rpm
	0.974	0.966	0.968	0.917	0.990	Weighted Avg.
<i>MLP 2</i>	0.979	0.962	0.970	0.877	0.953	normal
	0.741	0.900	0.813	0.796	0.899	dos
	0.994	0.957	0.975	0.975	0.987	fuzzy
	1.000	0.900	0.947	0.946	0.955	gear
	1.000	1.000	1.000	1.000	1.000	rpm
	0.960	0.955	0.956	0.883	0.952	Weighted Avg.
<i>MLP 3</i>	0.998	0.957	0.977	0.911	0.989	normal
	0.740	1.000	0.851	0.845	0.982	dos
	0.997	0.934	0.964	0.964	0.984	fuzzy
	1.000	1.000	1.000	1.000	1.000	gear
	1.000	1.000	1.000	1.000	1.000	rpm
	0.974	0.965	0.967	0.916	0.989	Weighted Avg.
<i>MLP 4</i>	0.864	0.991	0.923	0.623	0.843	normal
	0.773	0.100	0.177	0.258	0.686	dos
	0.887	0.737	0.805	0.804	0.953	fuzzy
	0.950	0.700	0.806	0.807	0.860	gear
	1.000	0.800	0.889	0.889	0.915	rpm
	0.869	0.874	0.842	0.620	0.837	Weighted Avg.
<i>MLP 5</i>	0.766	1.000	0.868	0.000	0.491	normal
	0.000	0.000	0.000	0.000	0.536	dos
	0.000	0.000	0.000	0.000	0.483	fuzzy
	0.000	0.000	0.000	0.000	0.472	gear
	0.000	0.000	0.000	0.000	0.451	rpm
	0.587	0.766	0.665	0.000	0.492	Weighted Avg.

Moreover, once investigated the effectiveness of the proposed classification algorithms by building a model for each attack we consider a multi-label classification i.e., a model build with all the attacks: the aim is to demonstrate that the proposed solution can be generalized for different types of attacks.

Since MLP has best performances, we investigate if adding more layers we can obtain more interesting performances. In detail in this analysis we exploit six different deep learning configurations, varying hidden layer from 0 to 5. Feature vectors act as input (This is highlighted in green in Figure 3, and hidden layers are in red), the labels related to the analysed

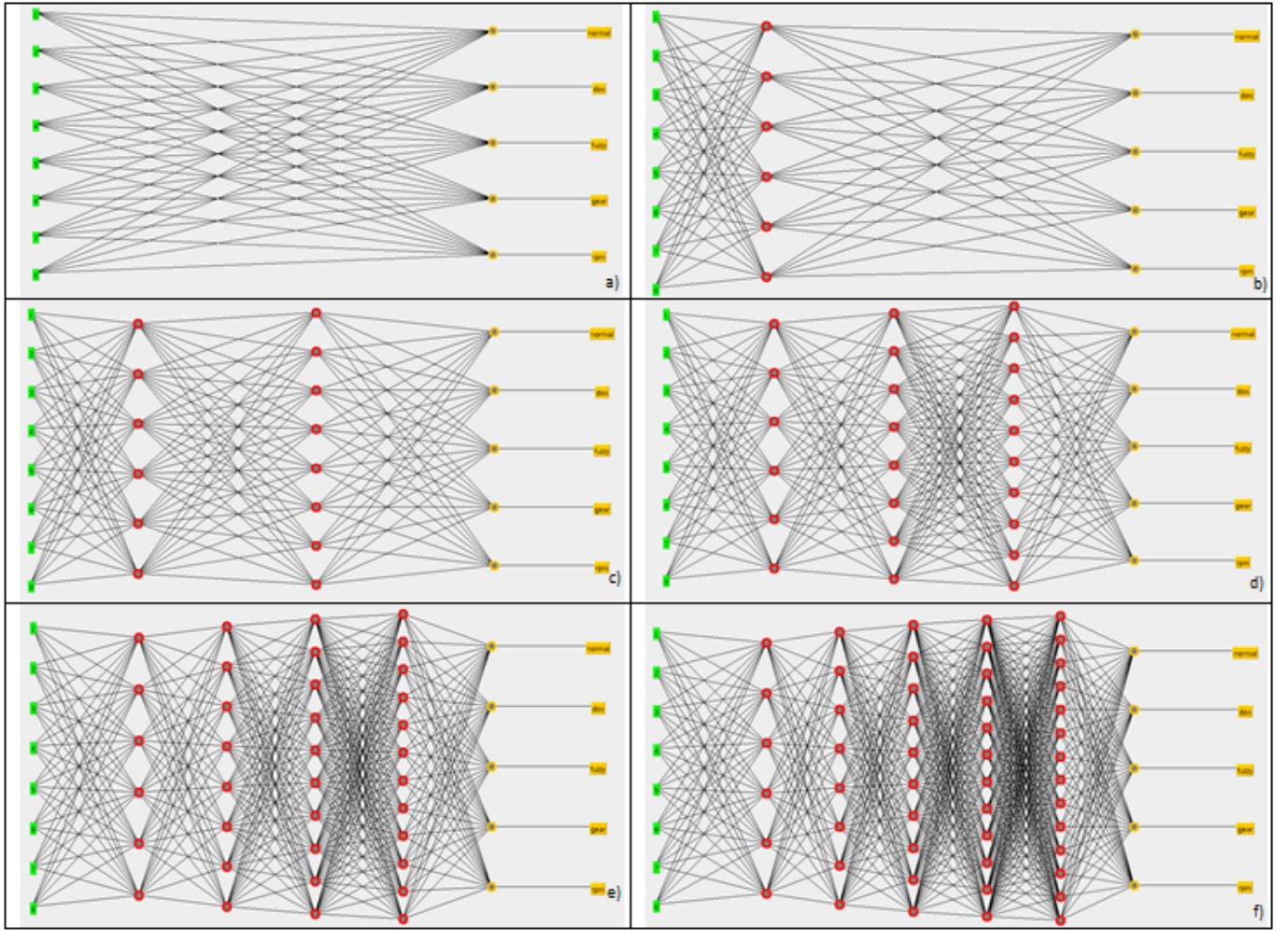


Fig. 3. The MLP networks considered in the second experiment. In green the input features, in red the hidden layers and in yellow the labels related to the possible predictions i.e., *dos*, *gear*, *rpm*, *fuzzy* and the *legitimate*.

attacks and to the normal messages (i.e., *dos*, *gear*, *rpm*, *fuzzy* and the *legitimate* class) represent the output layer (the yellow label in 3). Figure 3 resumes the network configuration we selected. Boxes from (a) to (f) reports configurations that uses from 0 to 5 hidden layers respectively.

Table V reports results of other seven experiments. The first one (NN) considers a network with 100 units and 1 hidden layer. The other ones consider MLP approaches. The numbers on each row represent the number of hidden layer in the experiment.

From the results of the second experiment, it is worth to note that all the MLP networks, with the exception of the MLP with 5 hidden layers, perform better than the NN model. In general, the weighted precision span between 0.587 by adopting the MLP 5 algorithm, up to 0.974 by using both the MLP 1 and the MLP 3 networks. Moreover, the average recall goes from 0.766 for the MLP 5 network to 0.966 of the MLP 1 network. At last, the results show that the performance decrease when more than 4 hidden states are added to the network. In fact, the precision and the recall are 0 for all the attacks in the MLP 5 network.

To conclude the MLP algorithm (in a configuration with no more than 3 hidden layers) exhibits better performances if

compared to the NN one. In fact the MLP with 3 hidden layers reached a weighed precision of 0.974 and a weighed recall equal to 0.965, demonstrating an interesting average ability to discriminate between legitimate traffic and malicious ones. With respect to the attack detection a precision and a recall equal to 1 is obtained relating to the detection of the malicious traffic when a *gear* or a *rpm* attack is in progress, while a precision (resp. recall) ranging from 0.740 (resp. 1) to 0.997 (resp. 0.943) is obtained respectively for the *dos* and the *fuzzy* attacks.

VI. CONCLUSION AND FUTURE WORK

A great amount of data is currently exchanged between the several electronic units currently employed in modern vehicles. To facilitate this exchange of messages the CAN-bus was introduced and from 1996 is mandatory. There are several weaknesses afflicting the CAN-bus for instance, no authentication is required to send or receive data and attacks, as for instance the denial of service, as demonstrated in literature, are easy to perpetrate. The automotive systems require highest safety levels, which is in contrast with the well-known vulnerabilities of the CAN-bus itself. For these reasons in this paper an approach aimed to detect attacks

against the CAN-bus in modern cars is proposed. We employ deep learning and we consider the CAN packet as feature vector. The main innovation introduced in this work, is that the approach aims at detecting the malicious injected messages by detecting the human behaviour of the attacker behind them. Experimental results on real-world dataset demonstrate the validity of the proposed approach in detecting Denial of Service attacks, fuzzy pattern attacks, and attacks against a specific components (i.e., gear and rpm). The algorithm shows the better results when the the MLP classification algorithm is adopted to train the network. In particular, with 1 and 3 hidden layers it is able to reach a weighted precision higher than 97% and a weighted recall higher than 96.5% in both the cases.

As a future work, the plan is to validate the proposed network with an extended set of attacks of a different nature. Moreover, we plan to adopt formal verification techniques with the aim to improve the obtained performances. Furthermore, we will consider ensemble learning to try to improve the classification performance, consisting in a series of classifiers aimed to obtain better predictive performance than the single model.

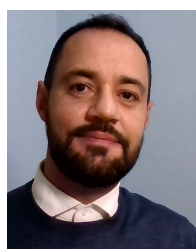
REFERENCES

- [1] F. Martinelli, F. Mercaldo, V. Nardone, and A. Santone, "Car hacking identification through fuzzy logic algorithms," in *Proc. IEEE Int. Conf. Fuzzy Syst. (FUZZ-IEEE)*, Jul. 2017, pp. 1–7.
- [2] K. Koscher *et al.*, "Experimental security analysis of a modern automobile," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 447–462.
- [3] F. Martinelli, F. Mercaldo, V. Nardone, A. Orlando, and A. Santone, "Who's driving my car? A machine learning based approach to driver identification," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 367–372.
- [4] B. I. Kwak, J. Woo, and H. K. Kim, "Know your master: Driver profiling-based anti-theft method," in *Proc. 14th Annu. Conf. Privacy, Secur. Trust (PST)*, Dec. 2016, pp. 211–218.
- [5] M. S. Al-kahtani, "Survey on security attacks in vehicular ad hoc networks (VANETs)," in *Proc. 6th Int. Conf. Signal Process. Commun. Syst.*, Dec. 2012, pp. 1–9.
- [6] G. Samara, W. A. Al-Salihy, and R. Sures, "Security issues and challenges of vehicular ad hoc networks (VANET)," in *Proc. 4th Int. Conf. New Trends Inf. Sci. Service Sci. (NISS)*, May 2010, pp. 393–398.
- [7] M. Wolf, A. Weimerskirch, and C. Paar, "Security in automotive bus systems," in *Proc. Workshop Embedded Secur. Cars*, 2004, pp. 1–13.
- [8] F. Martinelli, F. Mercaldo, A. Orlando, V. Nardone, A. Santone, and A. K. Sangaiah, "Human behavior characterization for driving style recognition in vehicle system," *Comput. Electr. Eng.*, vol. 83, May 2020, Art. no. 102504.
- [9] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Netw.*, vol. 61, pp. 85–117, Jan. 2015.
- [10] L. Deng and D. Yu, "Deep learning: Methods and applications," *Found. Trends Signal Process.*, vol. 7, nos. 3–4, pp. 197–387, Jun. 2014.
- [11] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 3104–3112.
- [12] G. Villarrubia, J. F. D. Paz, P. Chamoso, and F. D. L. Prieta, "Artificial neural networks used in optimization problems," *Neurocomputing*, vol. 272, pp. 10–16, Jan. 2018.
- [13] S. Checkoway *et al.*, "Comprehensive experimental analyses of automotive attack surfaces," in *Proc. 20th USENIX Secur. Symp. (USENIX Secur.)*, San Francisco, CA, USA, vol. 4, 2011, pp. 447–462.
- [14] S. Woo, H. J. Jo, and D. H. Lee, "A practical wireless attack on the connected car and security protocol for in-vehicle CAN," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 993–1006, Apr. 2015.
- [15] A. Taylor, S. Leblanc, and N. Japkowicz, "Anomaly detection in automobile control network data with long short-term memory networks," in *Proc. IEEE Int. Conf. Data Sci. Adv. Anal. (DSAA)*, Oct. 2016, pp. 130–139.
- [16] N. Lyamin, A. Vinel, M. Jonsson, and J. Loo, "Real-time detection of denial-of-service attacks in IEEE 802.11p vehicular networks," *IEEE Commun. Lett.*, vol. 18, no. 1, pp. 110–113, Jan. 2014.
- [17] K. M. A. Alheeti, A. Gruebler, and K. D. McDonald-Maier, "An intrusion detection system against malicious attacks on the communication network of driverless cars," in *Proc. 12th Annu. IEEE Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2015, pp. 916–921.
- [18] W. Liu, H. Zhang, and W. Zhang, "An autonomous road side infrastructure based system in secure VANETs," in *Proc. 5th Int. Conf. Wireless Commun., Netw. Mobile Comput.*, Sep. 2009, pp. 1–6.
- [19] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198.
- [20] S. Tariq, S. Lee, and S. S. Woo, "CANTransfer: Transfer learning based intrusion detection on a controller area network using convolutional LSTM network," in *Proc. 35th Annu. ACM Symp. Appl. Comput.*, Mar. 2020, pp. 1048–1055.
- [21] P. Golle, D. Greene, and J. Staddon, "Detecting and correcting malicious data in VANETs," in *Proc. 1st ACM Workshop Veh. Ad Hoc Netw. (VANET)*, 2004, pp. 29–37.
- [22] G. Yan, S. Olariu, and M. C. Weigle, "Providing VANET security through active position detection," *Comput. Commun.*, vol. 31, no. 12, pp. 2883–2897, Jul. 2008.
- [23] F. Martinelli, F. Mercaldo, and A. Santone, "Machine learning for driver detection through CAN bus," in *Proc. IEEE 91st Veh. Technol. Conf. (VTC-Spring)*, May 2020, pp. 1–5.
- [24] M. Hanselmann, T. Strauss, K. Dormann, and H. Ulmer, "CANet: An unsupervised intrusion detection system for high dimensional CAN bus data," *IEEE Access*, vol. 8, pp. 58194–58205, 2020.
- [25] N. Jargalsaikhan and K. Matsuyama, "An investigation of machine learning methods for prediction bus travel time of Mongolian public transportation," *Proc. SPIE*, vol. 11515, Jun. 2020, Art. no. 115151S.
- [26] K. Cheng, Y. Bai, Y. Zhou, Y. Tang, D. Sanan, and Y. Liu, "CANeleon: Protecting CAN bus with frame ID chameleon," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7116–7130, Jul. 2020.
- [27] D. Kosmanos *et al.*, "A novel intrusion detection system against spoofing attacks in connected electric vehicles," *Array*, vol. 5, Mar. 2020, Art. no. 100013.
- [28] M. Baccouche, F. Mamalet, C. Wolf, C. Garcia, and A. Baskurt, "Sequential deep learning for human action recognition," in *Proc. Int. Workshop Hum. Behav. Understand.* Berlin, Germany: Springer, 2011, pp. 29–39.
- [29] L. Ding, W. Fang, H. Luo, P. E. D. Love, B. Zhong, and X. Ouyang, "A deep hybrid learning model to detect unsafe behavior: Integrating convolution neural networks and long short-term memory," *Autom. Construct.*, vol. 86, pp. 118–124, Feb. 2018.
- [30] H. Rahmani, A. Mian, and M. Shah, "Learning a deep model for human action recognition from novel viewpoints," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 3, pp. 667–681, Mar. 2018.



Flora Amato

Flora Amato is currently a Tenure-track Assistant Professor with the Department of Electrical Engineering and Information Technology, Faculty of Engineering, University of Napoli Federico II, where she carries out her research activity since 2006. She is also a Professor of Artificial Intelligence for the degree in Computer Science Engineering with the University of Naples Federico II. She is author of more of 150 research articles, published on International journals and Conference Proceedings. Her research interests include knowledge management, information integration, text mining, and formal modeling.



Luigi Coppolino

Luigi Coppolino is currently an Associate Professor with the University of Naples Parthenope. His research interests include dependability of computing systems, critical infrastructure protection, and information security. His research has received massive funding from the European Union, within the context of the Seventh Framework Programme (FP7) and of Horizon 2020 (H2020).



Francesco Mercaldo received the master's degree in computer engineering from the University of Sannio, Benevento, Italy, with a thesis in software testing, and the Ph.D. degree with a dissertation on malware analysis using machine learning techniques, in 2015. He is currently working as a Researcher with the University of Molise, Campobasso, Italy. His research interests include software testing, verification, and validation, with the emphasis on the application of empirical methods.



Roberto Nardone received the master's degree in computer engineering and the Ph.D. degree in computer and automation engineering from the University of Naples Federico II, Italy, in 2009 and 2013, respectively. Since 2018, he has been an Assistant Professor with the Mediterranean University of Reggio Calabria, Italy. He has coauthored more than 60 research articles published in peer-reviewed international journals and conference proceedings. His research interest includes quantitative evaluation of non-functional properties by means of model-driven techniques. He serves as an Associate Editor for distinguished journals, such as IEEE ACCESS and *Journal of Universal Computer Science*.



Francesco Moscato received the degree (*summa cum laude*) in computer science engineering from the University of Naples Federico II, and the Ph.D. degree in electronic engineering from the Second University of Naples. He is currently an Associate Professor with the University of Salerno. He is author of many articles published on international journals and books. His research interests include formal modeling and verification of reliable and critical systems.



Antonella Santone received the Laurea degree in computer science from the University of Pisa, Italy, in April 1993, and the Ph.D. degree in computer systems engineering from the Dipartimento di Ingegneria della Informazione, University of Pisa, in September 1997. She was an Assistant Professor with the University of Pisa from November 1998 to October 2001. She was an Associate Professor with the Department of Engineering, University of Sannio, from November 2001 to August 2017. She has been an Associate Professor in computer engineering with the University of Molise, since September 2017. She was involved in several research activities and projects. Her current research interest includes formal verification methods. Her research interests include formal description techniques, temporal logic, concurrent and distributed systems modeling, heuristic search, and formal methods in systems biology and in software security. She has written more than 150 articles for international journals and conferences.